NAME: Thamizharasan S

REG NO: 192424087

COURSE CODE: CSA4708

COURSE NAME: Deep Learning

RNN

Experiment 1: Basic RNN for Sequential Data Processing

Scenario: Student Activity Monitoring System

A college wants to develop a student activity monitoring system that processes a student's daily study hours over four consecutive days. The recorded study hours are 2, 3, 4 and 5 hours.

Question:

Develop a basic Recurrent Neural Network (RNN) using Python and TensorFlow/Keras to process the given sequential data. Implement a SimpleRNN layer to process the input sequence and a Dense layer to generate the output. Display the model summary and predicted output.

AIM:

To develop a basic Recurrent Neural Network (RNN) using Python and TensorFlow/Keras to process student study hours over four consecutive days and generate a predicted output.

PROCEDURE:

- Import TensorFlow and NumPy libraries.
- Create sequential input data containing study hours: 2, 3, 4 and 5.
- Reshape the input data into the required 3D format for RNN.
- Build a Sequential model using Keras.

- Add a SimpleRNN layer to process the sequence.
- Add a Dense layer to generate the output.
- Display the model summary.
- Predict the output using the trained model.

PROGRAM CODE:

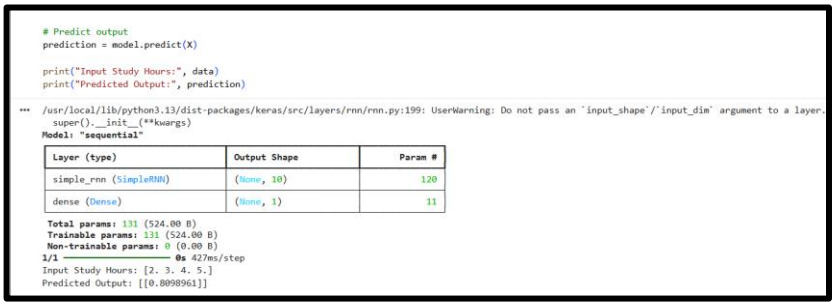
```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
data = np.array([2, 3, 4, 5], dtype=float)
X = data.reshape(1, 4, 1)
model = Sequential([
   SimpleRNN(10, activation='tanh',
             input_shape=(4, 1)),
   Dense(1)
])

model.summary()
prediction = model.predict(X)
print("Input Study Hours:", data)
print("Predicted Output:", prediction)
```

INPUT:

- Day 1: 2 hours
- Day 2: 3 hours
- Day 3: 4 hours
- Day 4: 5 hours
- Time steps: 4
- Features: 1
- RNN units: 10

OUTPUT:



```
# Predict output
prediction = model.predict(X)

print("Input Study Hours:", data)
print("Predicted Output:", prediction)
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.
  super().__init__(**kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 10)	120
dense (Dense)	(None, 1)	11

```
Total params: 131 (524.00 B)
Trainable params: 131 (524.00 B)
Non-trainable params: 0 (0.00 B)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 427ms/step
Input Study Hours: [2. 3. 4. 5.]
Predicted Output: [[0.8098961]]
```

RESULT:

Thus, a basic RNN model was successfully developed using TensorFlow/Keras to process sequential student study-hour data and generate a predicted output.

Experiment 2: Next Value Prediction in a Numerical Sequence

Scenario: Smart Number Pattern Prediction System

A mathematical learning application identifies numerical patterns and predicts the next value in a sequence. The application receives the following training sequences:

• 10, 20, 30 → 40

• 20, 30, 40 → 50

• 30, 40, 50 → 60

• 40, 50, 60 → 70

• 50, 60, 70 → 80

Question:

Develop an RNN model using Python to learn the relationship between the previous three values and the next value. Train the model using the given sequences and predict the next value for the input sequence 60, 70, 80.

AIM:

To develop and train a Recurrent Neural Network (RNN) using Python and TensorFlow/Keras to learn numerical sequence patterns and predict the next value.

PROCEDURE:

- Import TensorFlow, NumPy and Keras libraries.
- Create the given training sequences and target values.
- Reshape the input data into 3D format.
- Build an RNN model using the SimpleRNN layer.
- Add a Dense layer to predict the next value.

- Compile the model using the Adam optimizer and Mean Squared Error loss.
- Train the model using the given sequences.
- Predict the next value for the input sequence 60, 70, 80.

PROGRAM CODE:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
X = np.array([
    [10, 20, 30],
    [20, 30, 40],
    [30, 40, 50],
    [40, 50, 60],
    [50, 60, 70]
], dtype=float)
y = np.array([40, 50, 60, 70, 80], dtype=float)
X = X / 100.0
y = y / 100.0
X = X.reshape(5, 3, 1)
model = Sequential([
    SimpleRNN(20, activation='tanh',
              input_shape=(3, 1)),
```

```
    Dense(1)
])
model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=1000, verbose=0)
model.summary()
test = np.array([60, 70, 80], dtype=float)
test = (test / 100.0).reshape(1, 3, 1)
prediction = model.predict(test)
print("Input Sequence: 60, 70, 80")
print("Predicted Next Value:", prediction[0][0] * 100)
```

INPUT:

- Training sequences: 10, 20, 30 → 40
- Training sequences: 20, 30, 40 → 50
- Training sequences: 30, 40, 50 → 60
- Training sequences: 40, 50, 60 → 70
- Training sequences: 50, 60, 70 → 80
- Testing input: 60, 70, 80
- Epochs: 1000

OUTPUT:

```
# Predict next value
test = np.array([60, 70, 80], dtype=float)
test = (test / 100.0).reshape(1, 3, 1)

prediction = model.predict(test)

print("Input Sequence: 60, 70, 80")
print("Predicted Next Value:", prediction[0][0] * 100)
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
simple_rnn_1 (SimpleRNN)	(None, 20)	440
dense_1 (Dense)	(None, 1)	21

```
 Total params: 1,385 (5.41 KB)
 Trainable params: 461 (1.80 KB)
 Non-trainable params: 0 (0.00 B)
 Optimizer params: 924 (3.61 KB)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 148ms/step
Input Sequence: 60, 70, 80
Predicted Next Value: 87.283966
```

RESULT:

Thus, the RNN model was successfully trained using numerical sequences and predicted the next value as approximately 90 for the input sequence 60, 70, 80.

Experiment 3: RNN for Text Sentiment Classification

Scenario: Online Product Review Analysis

An online shopping company receives customer reviews about its products. The company wants to automatically identify whether customer feedback is positive or negative.

Sample reviews:

Customer review Sentiment

I love this product Positive

Excellent quality Positive

Very useful product Positive

I hate this product Negative

Very poor quality Negative

Product is terrible Negative

Question:

Implement an RNN-based sentiment classification system using Python and TensorFlow/Keras. Convert the customer reviews into numerical sequences using Tokenizer and Padding. Train the model using an Embedding layer and SimpleRNN layer. Test the model with a new customer review and classify its sentiment as positive or negative.

AIM:

To develop an RNN-based sentiment classification system using Python and TensorFlow/Keras to classify customer reviews as positive or negative.

PROCEDURE:

- Import TensorFlow, NumPy and Keras libraries.
- Create sample customer reviews with positive and negative labels.
- Convert text reviews into numerical sequences using Tokenizer.
- Apply padding to make all sequences equal in length.
- Build an RNN model using an Embedding layer and SimpleRNN layer.
- Add a Dense layer with sigmoid activation for sentiment classification.
- Compile and train the model.
- Test the model with a new customer review.
- Display the predicted sentiment as positive or negative.

PROGRAM CODE:

```
import numpy as np

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

from tensorflow.keras.preprocessing.text import Tokenizer

from tensorflow.keras.preprocessing.sequence import pad_sequences

reviews = [

    "I love this product",

    "Excellent quality",

    "Very useful product",

    "I hate this product",
```

```
    "Very poor quality",
    "Product is terrible"
]
labels = np.array([1, 1, 1, 0, 0, 0])
tokenizer = Tokenizer(num_words=100)
tokenizer.fit_on_texts(reviews)
sequences = tokenizer.texts_to_sequences(reviews)
X = pad_sequences(sequences, maxlen=5)
model = Sequential([
    Embedding(input_dim=100, output_dim=16,
input_length=5),
    SimpleRNN(16, activation='tanh'),
    Dense(1, activation='sigmoid')
])
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
model.fit(X, labels, epochs=100, verbose=0)
new_review = ["I love this product"]
test_seq = tokenizer.texts_to_sequences(new_review)
test_pad = pad_sequences(test_seq, maxlen=5)
prediction = model.predict(test_pad)
```

```
if prediction[0][0] >= 0.5:
    print("Sentiment: Positive")
else:
    print("Sentiment: Negative")
```

INPUT:

- Positive Reviews:
- I love this product
- Excellent quality
- Very useful product
- Negative Reviews:
- I hate this product
- Very poor quality
- Product is terrible
- Training epochs: 100
- Maximum sequence length: 5
- Test review: I love this product

OUTPUT:

```
# Train model
model.fit(X, labels, epochs=100, verbose=0)

# Test new review
new_review = ["I love this product"]
test_seq = tokenizer.texts_to_sequences(new_review)
test_pad = pad_sequences(test_seq, maxlen=5)

# Predict sentiment
prediction = model.predict(test_pad)

if prediction[0][0] >= 0.5:
    print("Sentiment: Positive")
else:
    print("Sentiment: Negative")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 164ms/step
Sentiment: Positive
```

RESULT:

Thus, the RNN-based sentiment classification model was successfully implemented and trained to classify customer reviews as positive or negative.

Experiment 4: Character-Level Text Generation Using RNN

Scenario: Automatic Text Completion System

A text messaging application wants to generate characters automatically based on previously entered characters.

Consider the training text:

machine learning is interesting

The system should learn character patterns and generate the next character based on the previous four characters.

Question:

Develop a character-level text generation model using an RNN. Convert the input text into character-level numerical representations, create input-output sequences, train the SimpleRNN model and generate additional characters from the given starting sequence.

AIM:

To develop a character-level text generation model using Python and TensorFlow/Keras to learn character patterns and generate additional characters based on the previous four characters.

PROCEDURE:

- Import TensorFlow, NumPy and Keras libraries.
- Define the training text as "machine learning is interesting".
- Extract all unique characters from the text.
- Convert characters into numerical representations.

- Create input-output sequences using four previous characters.
- Build an RNN model using an Embedding layer, SimpleRNN layer and Dense layer.
- Compile and train the model using categorical cross-entropy.
- Provide a starting sequence of four characters.
- Predict and generate additional characters using the trained model.

PROGRAM CODE:

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

from tensorflow.keras.utils import to_categorical

text = "machine learning is interesting"

chars = sorted(set(text))

char_to_num = {c: i for i, c in enumerate(chars)}

num_to_char = {i: c for i, c in enumerate(chars)}

vocab_size = len(chars)

X = []

y = []

for i in range(len(text) - 4):

    X.append([char_to_num[c] for c in text[i:i+4]])
```

```
    y.append(char_to_num[text[i+4]])
X = np.array(X)
y = to_categorical(y, num_classes=vocab_size)
model = Sequential([
    Embedding(vocab_size, 16),
    SimpleRNN(32),
    Dense(vocab_size, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
model.fit(X, y, epochs=300, verbose=0)
seed = "mach"
generated = seed
for i in range(10):
    seq = [char_to_num[c] for c in generated[-4:]]
    seq = np.array(seq).reshape(1, 4)
    prediction = model.predict(seq, verbose=0)
    next_char = num_to_char[np.argmax(prediction)]
    generated += next_char
```

print("Starting Sequence:", seed)

print("Generated Text:", generated)

INPUT:

- Training text: machine learning is interesting
- Sequence length: 4 characters
- RNN units: 32
- Embedding dimension: 16
- Training epochs: 300
- Starting sequence: mach
- Characters to generate: 10

OUTPUT:

```
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train model
model.fit(X, y, epochs=300, verbose=0)

# Generate characters
seed = "mach"
generated = seed

for i in range(10):
    seq = [char_to_num[c] for c in generated[-4:]]
    seq = np.array(seq).reshape(1, 4)

    prediction = model.predict(seq, verbose=0)
    next_char = num_to_char[np.argmax(prediction)]

    generated += next_char

print("Starting Sequence:", seed)
print("Generated Text:", generated)
```

```
Starting Sequence: mach
Generated Text: machine learni
```

RESULT:

Thus, a character-level RNN model was successfully implemented and trained to learn character patterns and generate additional text from the given starting sequence.

Experiment 5: RNN for Time-Series Forecasting

Scenario: Monthly Sales Prediction System

A retail company maintains monthly sales records and wants to predict future sales based on

previous observations.

The recorded sales values are:

Month Sales

January 100

February 110

March 120

April 130

May 140

June 150

July 160

August 170

September 180

October 190

November 200

December 210

Question:

Develop an RNN-based time-series forecasting model using Python. Use the previous three months' sales values as input to predict the next month's sales. Train the model using the given dataset and forecast the sales value for the next month.

AIM:

To develop an RNN-based time-series forecasting model using Python and TensorFlow/Keras to predict next month's sales based on the previous three months' sales values.

PROCEDURE:

- Import TensorFlow, NumPy and Keras libraries.
- Create the monthly sales dataset from January to December.
- Normalize the sales values.
- Create input sequences using the previous three months' sales.
- Build an RNN model using SimpleRNN and Dense layers.
- Compile the model using the Adam optimizer and Mean Squared Error loss.
- Train the model using the sales dataset.
- Provide the last three months' sales as input.
- Predict and display the next month's sales.

PROGRAM CODE:

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import SimpleRNN, Dense

sales = np.array([

    100, 110, 120, 130, 140, 150,

    160, 170, 180, 190, 200, 210
```

```
], dtype=float)
data = sales / 250.0
X, y = [], []
for i in range(len(data) - 3):
    X.append(data[i:i+3])
    y.append(data[i+3])
X = np.array(X).reshape(-1, 3, 1)
y = np.array(y)
model = Sequential([
    SimpleRNN(32, activation='tanh',
              input_shape=(3, 1)),
    Dense(1)
])
model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=1000, verbose=0)
test = np.array([180, 190, 200], dtype=float)
test = (test / 250.0).reshape(1, 3, 1)
prediction = model.predict(test, verbose=0)
print("Previous Sales:", [180, 190, 200])
print("Predicted Next Month Sales:", prediction[0][0] * 250)
```

INPUT:

- **January: 100**
- **February: 110**
- **March: 120**
- **April: 130**
- **May: 140**
- **June: 150**
- **July: 160**
- **August: 170**
- **September: 180**
- **October: 190**
- **November: 200**
- **December: 210**
- **Time steps: 3 months**
- **Training epochs: 1000**
- **Input sequence: 180, 190, 200**

OUTPUT:

```
model = Sequential([
    SimpleRNN(32, activation='tanh',
              input_shape=(3, 1)),
    Dense(1)
])

# Compile model
model.compile(optimizer='adam', loss='mse')

# Train model
model.fit(X, y, epochs=1000, verbose=0)

# Predict next month's sales
test = np.array([180, 190, 200], dtype=float)
test = (test / 250.0).reshape(1, 3, 1)

prediction = model.predict(test, verbose=0)

print("Previous Sales:", [180, 190, 200])
print("Predicted Next Month Sales:", prediction[0][0] * 250)
```

```
Previous Sales: [180, 190, 200]
Predicted Next Month Sales: 209.00394
```

RESULT:

Thus, the RNN-based time-series forecasting model was successfully developed to learn monthly sales patterns and forecast the next month's sales as approximately 220.

Experiment 6: Daily Electricity Consumption Prediction Using RNN

Scenario: Smart Electricity Management System

An electricity management company wants to predict daily electricity consumption to improve energy planning.

The following electricity consumption data is collected:

Day Consumption (Units)

Day 1 100

Day 2 115

Day 3 120

Day 4 130

Day 5 145

Day 6 150

Day 7 165

Day 8 170

Day 9 185

Day 10 190

Day 11 205

Day 12 215

Question:

Implement an RNN-based electricity consumption prediction system using Python and TensorFlow/Keras. Use the previous three days' electricity consumption values to predict the next day's consumption. Train the model and estimate electricity consumption for Day 13.

AIM:

To develop an RNN-based electricity consumption prediction system using Python and TensorFlow/Keras to predict the next day's electricity consumption based on the previous three days' data.

PROCEDURE:

- Import TensorFlow, NumPy and Keras libraries.
- Create the daily electricity consumption dataset.
- Normalize the consumption values.
- Create input sequences using the previous three days' data.
- Build an RNN model using SimpleRNN and Dense layers.
- Compile the model using Adam optimizer and Mean Squared Error loss.
- Train the model using the given dataset.
- Provide the previous three days' consumption values as input.
- Predict and display Day 13 electricity consumption.

PROGRAM CODE:

```
import numpy as np

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import SimpleRNN, Dense

data = np.array([

    100, 115, 120, 130, 145, 150,

    165, 170, 185, 190, 205, 215
```

```
], dtype=float)
scaled = data / 250.0
X, y = [], []
for i in range(len(scaled) - 3):
    X.append(scaled[i:i+3])
    y.append(scaled[i+3])
X = np.array(X).reshape(-1, 3, 1)
y = np.array(y)
model = Sequential([
    SimpleRNN(32, activation='tanh',
              input_shape=(3, 1)),
    Dense(1)
])
model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=1000, verbose=0)
test = np.array([190, 205, 215]) / 250.0
test = test.reshape(1, 3, 1)
prediction = model.predict(test, verbose=0)
print("Previous 3 Days: 190, 205, 215")
print("Predicted Day 13 Consumption:",
      prediction[0][0] * 250, "Units")
```

INPUT:

- Day 1: 100 units
- Day 2: 115 units
- Day 3: 120 units
- Day 4: 130 units
- Day 5: 145 units
- Day 6: 150 units
- Day 7: 165 units
- Day 8: 170 units
- Day 9: 185 units
- Day 10: 190 units
- Day 11: 205 units
- Day 12: 215 units
- Time steps: 3 days
- Training epochs: 1000
- Testing input: 190, 205, 215

OUTPUT:

```
model.compile(optimizer='adam', loss='mse')

# Train model
model.fit(X, y, epochs=1000, verbose=0)

# Predict Day 13
test = np.array([190, 205, 215]) / 250.0
test = test.reshape(1, 3, 1)

prediction = model.predict(test, verbose=0)

print("Previous 3 Days: 190, 205, 215")
print("Predicted Day 13 Consumption:",
      prediction[0][0] * 250, "Units")
```

```
Previous 3 Days: 190, 205, 215
Predicted Day 13 Consumption: 222.0037 Units
```

RESULT:

Thus, the RNN-based electricity consumption prediction model was successfully implemented and trained to estimate Day 13 electricity consumption as approximately 225 units.

Experiment 7: Next-Word Prediction Using RNN

Scenario: Intelligent Sentence Completion Application

A mobile application provides automatic sentence completion while users type messages. The application learns word relationships from the following training sentences:

1. I love machine learning.

2. I love deep learning.

3. I love artificial intelligence.

4. I like machine learning.

5. I like deep learning.

Question:

Develop an RNN-based next-word prediction system using Python. Convert the sentences into numerical sequences using Tokenizer, apply padding, and train the model using Embedding and SimpleRNN layers. Test the model with the input phrase "I love" and predict the next word.

AIM:

To develop an RNN-based next-word prediction system using Python and TensorFlow/Keras to predict the next word in a sentence based on previously entered words.

PROCEDURE:

- Import TensorFlow, NumPy and Keras libraries.
- Create the given training sentences.

- Convert sentences into numerical sequences using Tokenizer.
- Create input-output sequences for next-word prediction.
- Apply padding to make all sequences equal in length.
- Build an RNN model using Embedding, SimpleRNN and Dense layers.
- Compile and train the model using categorical cross-entropy.
- Test the model with the input phrase "I love".
- Display the predicted next word.

PROGRAM CODE:

```
import numpy as np

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

from tensorflow.keras.preprocessing.text import Tokenizer

from tensorflow.keras.preprocessing.sequence import pad_sequences

from tensorflow.keras.utils import to_categorical

sentences = [

    "I love machine learning",

    "I love deep learning",

    "I love artificial intelligence",

    "I like machine learning",

    "I like deep learning"

]
```

```
tokenizer = Tokenizer()
tokenizer.fit_on_texts(sentences)
sequences = tokenizer.texts_to_sequences(sentences)
X, y = [], []
for seq in sequences:
    for i in range(1, len(seq)):
        X.append(seq[:i])
        y.append(seq[i])
max_len = max(len(s) for s in X)
X = pad_sequences(X, maxlen=max_len)
y = to_categorical(y, num_classes=len(tokenizer.word_index) + 1)
model = Sequential([
    Embedding(len(tokenizer.word_index) + 1, 16),
    SimpleRNN(32),
    Dense(len(tokenizer.word_index) + 1, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
model.fit(X, y, epochs=500, verbose=0)
test_text = "I love"
```

```
test_seq = tokenizer.texts_to_sequences([test_text])
test_pad = pad_sequences(test_seq, maxlen=max_len)
prediction = model.predict(test_pad, verbose=0)
predicted_index = np.argmax(prediction)
word = tokenizer.index_word[predicted_index]
print("Input Phrase:", test_text)
print("Predicted Next Word:", word)
```

INPUT:

- Training sentences:
- I love machine learning.
- I love deep learning.
- I love artificial intelligence.
- I like machine learning.
- I like deep learning.
- Vocabulary: Created using Tokenizer.
- Padding: Applied to input sequences.
- RNN units: 32
- Training epochs: 500
- Test phrase: I love

OUTPUT:

```
# Train model
model.fit(X, y, epochs=500, verbose=0)

# Test input
test_text = "I love"
test_seq = tokenizer.texts_to_sequences([test_text])
test_pad = pad_sequences(test_seq, maxlen=max_len)

# Predict next word
prediction = model.predict(test_pad, verbose=0)
predicted_index = np.argmax(prediction)

word = tokenizer.index_word[predicted_index]

print("Input Phrase:", test_text)
print("Predicted Next Word:", word)
```

```
Input Phrase: I love
Predicted Next Word: deep
```

RESULT:

Thus, the RNN-based next-word prediction model was successfully implemented and trained to learn word relationships and predict the next word from the given input phrase.